Задание для Продакт-оунера
Цель: сформировать список функциональных требований, расставить приоритеты и выделить MVP.

Что сделать:

Составить реестр требований. На базе ранее выделенных типов пользователей (студент, преподаватель, методист) выпиши не менее 15 требований в формате «Система должна…». Пример: «Система должна позволять студенту искать курс по названию дисциплины и семестру».
Расставить приоритеты по методу MoSCoW. Раздели требования на 4 группы:

Must have (обязательно для MVP): без этого продукт не имеет смысла (например, каталог с карточками курсов).

Should have (желательно): важно, но можно перенести на следующий релиз (например, расширенная аналитика для администрации).

Could have (возможно): nice-to-have, если останется время (например, личный кабинет студента с прогрессом).

Won’t have (не делаем сейчас): явно откладываем (например, мобильное приложение).
Определить границы MVP. Сформулируй 3–5 ключевых функций, которые будут реализованы в первой версии. Для каждой укажи:
какую проблему пользователя решает;
какой измеримый результат ожидается (например, «студент находит РПД за ≤30 секунд»).

Оформить результат. Представь итог в виде таблицы (требование | тип пользователя | приоритет | обоснование) и

краткого абзаца (2–3 предложения) «Что такое MVP Training Library в этой версии».

Задание для Разработчика
Цель: оценить сложность реализации, выявить технические ограничения и зависимости.
Что сделать:

Оценить требования из задания продакт-оунера. Для каждого пункта из группы «Must have» дай оценку сложности (низкая/средняя/высокая) и трудоёмкости (в условных единицах или часах). Кратко (1–2 предложения) обоснуй оценку (например, «средняя — нужна интеграция с 1С, API нестабильное»).

Выявить технические ограничения. На базе ранее обсуждённых ограничений (инфраструктура колледжа, 152-ФЗ, старые браузеры) выбери 3–5 самых критичных для MVP. Для каждого оформи карточку:
ограничение (например, «данные только на серверах в РФ»);
влияние на архитектуру (например, «исключаем AWS, выбираем Yandex Cloud»);

решение (например, «развёртывание в облаке Yandex, шифрование дисков»).

Определить зависимости. Составь список внешних систем и внутренних ролей, от которых зависит реализация (например, «доступ к API 1С», «согласование ролей с администрацией»). Для каждой зависимости укажи:
тип (внешняя система / человек / норматив);
риск (низкий/средний/высокий);
срок, когда зависимость должна быть закрыта.

Оформить результат. Представь итог в виде двух списков (оценка требований и ограничения) и одной таблицы (зависимости). Добавь 1 абзац (3–5 предложений) с итоговым выводом: «MVP реализуем за X недель при условии Y».

Задание для Дизайнер-аналитика
Цель: описать пользовательские сценарии (User Flow) и определить экраны.
Что сделать:

Описать 3 ключевых User Flow (по одному на каждый тип пользователя) для сценариев из MVP:
Студент: «Найти и скачать РПД по дисциплине».
Преподаватель: «Загрузить новую версию РПД и отправить на согласование».

Методист: «Проверить список курсов на статусе “на согласовании” и утвердить/вернуть».
Для каждого потока опиши шаги (1–2 предложения на шаг), точки принятия решений и возможные ошибки (например, «курс не найден», «файл не прошёл валидацию»).
Определить список экранов. На базе User Flow выпиши все уникальные экраны, которые понадобятся. Сгруппируй их по ролям. Для каждого экрана укажи:

название (например, «Каталог курсов», «Карточка курса», «Админ-панель: очередь согласований»);
основные элементы (3–5 ключевых блоков/полей);
роль, у которой есть доступ.

Выделить общие компоненты. Определи 3–5 элементов, которые будут повторяться на разных экранах (например,

шапка с поиском, фильтр, карточка курса) — это поможет ускорить дизайн и разработку.

Оформить результат. Представь итог в виде 3 коротких сценариев (по 5–7 шагов каждый) и таблицы экранов (название | элементы | роль). Добавь 1 абзац (2–3 предложения) с рекомендацией: «С чего начать прототипирование (какой экран первый) и почему».